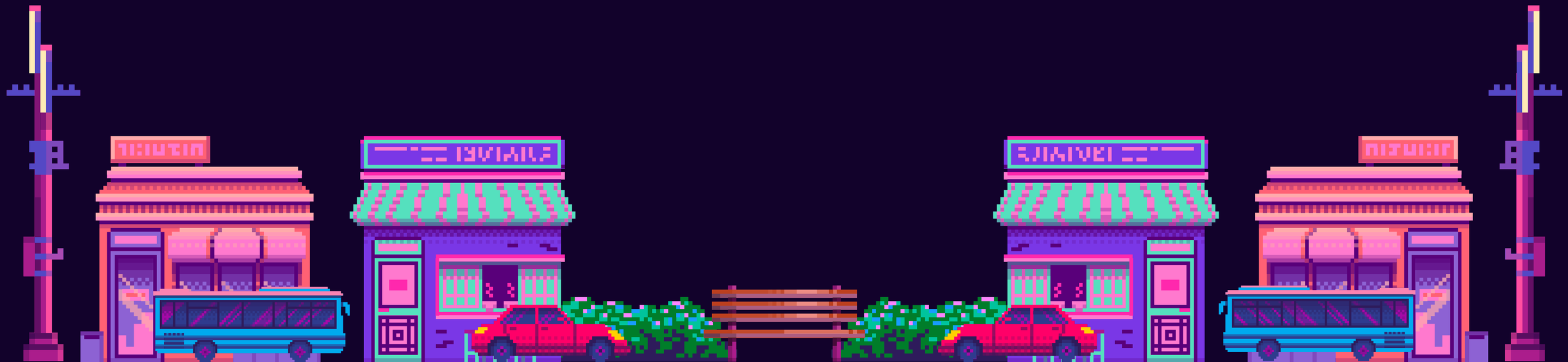
HELLO
WORLD

INTRODUÇÃO AO JAVASCRIPT

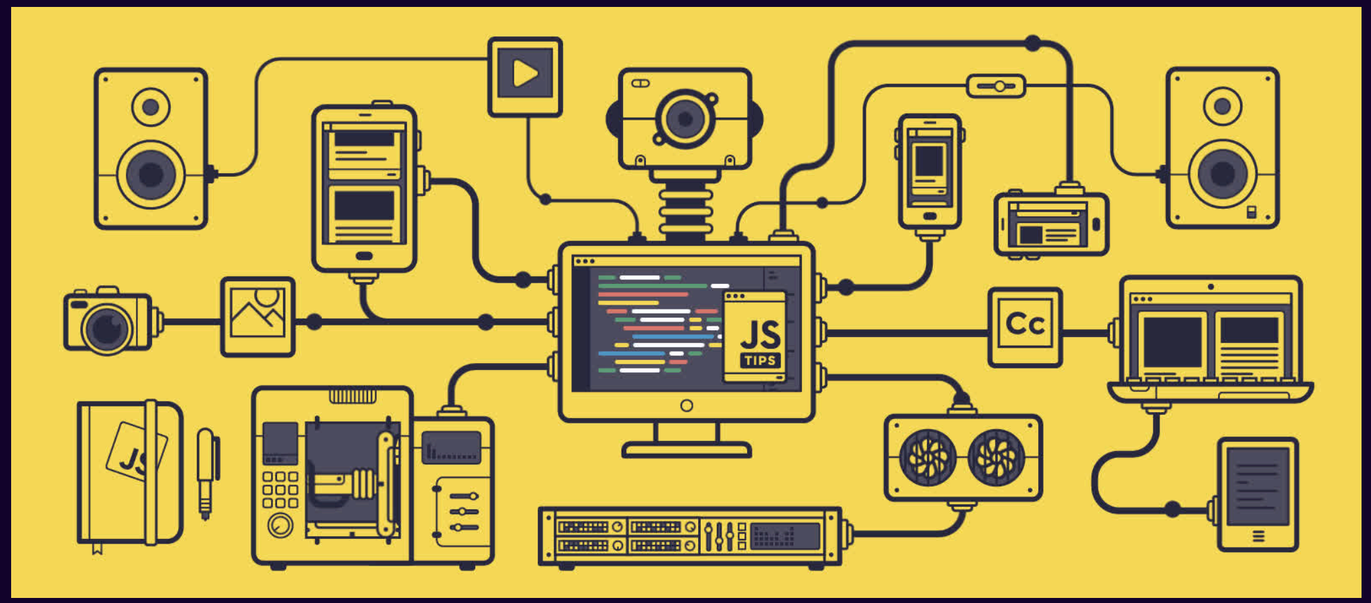


O QUE É O JAVASCRIPT?

JavaScript é uma linguagem de programação que adiciona interatividade ao seu site. Isso acontece em jogos, no comportamento das respostas quando botões são pressionados ou com entrada de dados em formulários; com estilo dinâmico; com animação, etc. Este artigo ajuda você a começar a usar o JavaScript e aumenta sua compreensão do que é possível.

JavaScript é uma poderosa linguagem de programação que pode adicionar interatividade a um site. Foi inventado por Brendan Eich. JavaScript é versátil e amigável para iniciantes. Com mais experiência, você poderá criar jogos, gráficos 2D e 3D animados, aplicativos abrangentes baseados em banco de dados e muito mais!

- Interfaces de programação de aplicativos de navegador (APIs) incorporadas a navegadores da Web, fornecendo funcionalidades como criação dinâmica de HTML e definição de estilos CSS; coletar e manipular um fluxo de vídeo da webcam de um usuário ou gerar gráficos 3D e amostras de áudio.
- APIs de terceiros que permitem aos desenvolvedores incorporar funcionalidades em sites de outros provedores de conteúdo, como Twitter ou Facebook.
- Estruturas e bibliotecas de terceiros que você pode aplicar ao HTML para acelerar o trabalho de construção de sites e aplicativos.



CONVENÇÕES NO JAVASCRIPT

- camelCase: Inicia com letra minúscula e a primeira letra de cada palavra em seguida é escrita com letra maiúscula. Por exemplo: minhaVar ou senhaDoUsuario. Esta é a convenção utilizada pelo JavaScript para variáveis e funções.

- snake_case: Os espaços são substituídos pelo caractere _ (underline), com todas as palavras em letra minúscula. Por exemplo: minha_variavel ou senha_do_usuario.
- kebab-case: Similar ao anterior, porém com os espaços substituídos por hífens. Por exemplo: minha-variavel ou senha-do-usuario. Esta convenção não pode ser utilizada no JavaScript para variáveis e funções.
- PascalCase: Similar ao camelCase , porém neste caso todas as palavras começam com letra maiúscula. Por exemplo: MinhaVar ou SenhaDoCliente.

```
{
    "camelCase": "camelCase",
    "PascalCase": "PascalCase",
    "snake_case": "snake_case",
    "kebab-case": "kebab-case"
}
```

VARIÁVEIS

Variables são contêineres que armazenam valores. Você começa declarando uma variável com a palavra-chave let, seguida do nome que você dá à variável:

```
let myVariable;
```

Depois de declarar uma variável, você pode atribuir um valor a ela:

```
myVariable = "Bob";
```

Além disso, você pode fazer essas duas operações na mesma linha:

```
let myVariable = "Bob";
```

Você recupera o valor chamando o nome da variável:

```
myVariable;
```

Depois de atribuir um valor a uma variável, você pode alterá-lo posteriormente no código:

```
let myVariable = "Bob";
myVariable = "Steve";
```

CONSTANTS

Nos primeiros dias do JavaScript, não existiam constants. No JavaScript moderno, temos a palavra-chave const, que nos permite armazenar valores que nunca podem ser alterados:

```
const diasNaSemana = 7;
const horasNoDia = 24;
```

const funciona exatamente da mesma maneira que let, exceto que você não pode atribuir um novo valor a const. No exemplo a seguir, a segunda linha geraria um erro:

```
const diasNaSemana = 7;
diasNaSemana = 8;
```

FUNÇÕES

Funções são blocos de código reutilizáveis que podem ser executados quando chamados. Em JavaScript, as funções podem ser definidas de várias maneiras, incluindo a sintaxe de função tradicional e as arrow functions (funções de seta). Vamos explorar esses conceitos em detalhes.

Funções Tradicionais

Uma função tradicional é definida usando a palavra-chave function. A sintaxe básica é a seguinte:

```
function nomeDaFuncao(param1, param2, ...) {
  // corpo da função
  return algumValor;
}
```

Arrow Functions

Introduzidas no ECMAScript 6 (ES6), as arrow functions são uma sintaxe mais curta para escrever funções anônimas. Elas são definidas usando a sintaxe de "seta" (=>). A sintaxe básica é:

```
const nomeDaFuncao = (param1, param2, ...) => {
  // corpo da função
  return algumValor;
};
```

O QUE É NODE.JS?

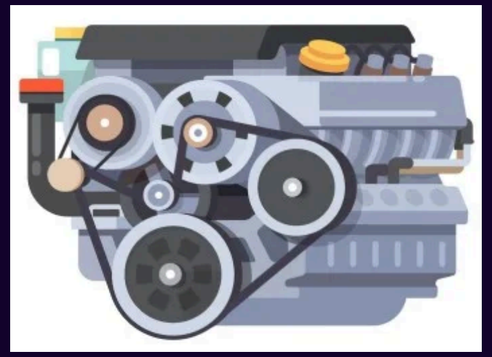
O Node.js é um ambiente de execução do código JavaScript do lado servidor (server side), que na prática se reflete na possibilidade de criar aplicações standalone (autossuficientes) em uma máquina servidora, sem a necessidade do navegador.

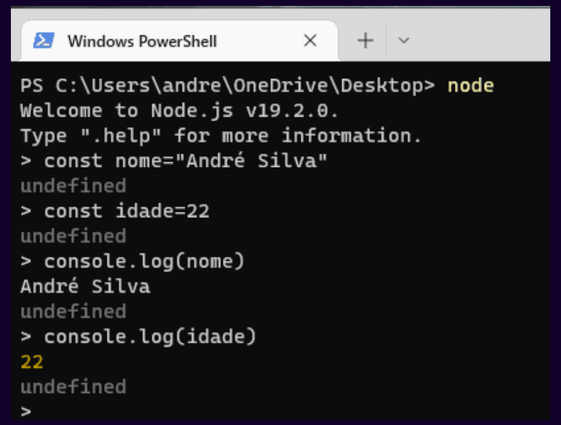
A História do Node.JS

O Node.js nasceu em 2009 como uma resposta às tentativas de rodar códigos JavaScript em modo server side, uma vez que a linguagem tinha como meta principal a manipulação do DOM (Document Object Model) e deixar as aplicações web mais interativas e dinâmicas.

JAVASCRIPT RUNTIME E V8

O Node.js é a ferramenta que vai nos entregar a capacidade de interpretar código JavaScript, de maneira bem similar ao navegador. Quando executamos um comando escrito em JavaScript, o Node.js interpreta esse comando e faz a sua conversão para a linguagem de máquina a ser executada pelo computador. Por esse motivo, o Node.js também pode ser referido como um JavaScript Runtime, ou um programa de execução do JavaScript.





Quando iniciamos o Node.js com o comando node, iniciamos um processo que engloba um interpretador JavaScript e um utilitário CLI (command-line interface), e é neste processo aberto no terminal que irá acontecer a interpretação e execução do JavaScript runtime. Para esse processo de interpretação, o Node.js faz uso do V8, mais precisamente conhecido como Chrome’s V8 JavaScript engine.

CARACTERÍSTICAS DO NODE.JS

No decorrer do artigo já falamos de algumas características do Node.js, mas vale ressaltar que ele pode ser usado para:

1. Desenvolver soluções em API Rest;
2. Criação de chatbots;
3. Projetos de internet das coisas (IoT); e
4. Dar vida a soluções de web scraping, web servers e até mesmo aplicações desktop.

O Node.js possui uma versatilidade gigantesca e Camila Pessôa, em seu artigo sobre como o Node.js funciona, elenca algumas outras características que não podemos deixar de comentar:

- Escalabilidade: o Node.js foi pensando desde o ínicio para entregar soluções para arquiteturas escaláveis, por isso sua adoção por grandes empresas de tecnologias;
- Multiplataforma: podemos criar desde soluções web a aplicações desktop;
- Open Source: como já citado, o Node.js tem seu código aberto, o que nos possibilita contribuir com o projeto e até mesmo realizar customizações;
- Multi-paradigma: com o JavaScript podemos adotar várias formas de codificar usando paradigmas como a orientação a objetos, o funcional e o orientado a eventos.

DOCUMENTAÇÃO DO NODE.JS

Node.js

About this documentation

Usage and example

Assertion testing

Asynchronous context tracking

Async hooks

Buffer

C++ addons

C/C++ addons with Node-API

C++ embedder API

Child processes

Cluster

Command-line options

Console

Corepack

Crypto

Debugger

Deprecated APIs

Diagnostics Channel

DNS

Domain

Errors

Events

File system

Node.js v22.5.1 documentation

▸ Other versions | ▸ Options

- About this documentation
- Usage and example

- Assertion testing
- Asynchronous context tracking
- Async hooks
- Buffer
- C++ addons
- C/C++ addons with Node-API
- C++ embedder API
- Child processes
- Cluster
- Command-line options
- Console
- Corepack
- Crypto
- Debugger
- Deprecated APIs
- Diagnostics Channel
- DNS
- Domain
- Errors
- Events
- File system
- Globals
- HTTP
- HTTP/2
- HTTPS
- Inspector

QUERY SELECTOR

querySelector é um método fornecido pela API do DOM (Document Object Model) em JavaScript. Ele é utilizado para selecionar o primeiro elemento que corresponde a um seletor CSS especificado. Este método é muito útil quando você deseja manipular elementos do DOM de forma precisa e específica, sem a necessidade de percorrer todos os elementos de uma página.

Sintaxe Básica

```
document.querySelector(seletor);
```

- document: Refere-se ao objeto global do DOM, representando toda a página web.
- seletor: Uma string que contém um ou mais seletores CSS. Pode ser uma classe, um ID, uma tag, um atributo, ou uma combinação desses.

Como Funciona

- O método querySelector retorna o primeiro elemento que corresponde ao seletor especificado. Se não houver correspondências, ele retorna null.

QUERY SELECTOR-EXEMPLOS DE USO

1. Selecionando por ID

```
const elemento = document.querySelector('#meuID');
```

2. Selecionando por Classe

```
const elemento = document.querySelector('.minhaClasse');
```

3. Selecionando por Tag

```
const elemento = document.querySelector('p');
```

4. Selecionando por Atributo

```
const elemento = document.querySelector('input[type="text"]');
```

5. Selecionando Elementos Aninhados

```
const elemento = document.querySelector('div > p');
```

BOAS PRÁTICAS

1. Use Nomes Descritivos para Variáveis e Funções:
Nomes significativos para variáveis e funções facilitam a compreensão do código. Evite abreviações obscuras e opte por nomes que expressem claramente a finalidade da variável ou função.

```
// Ruim
let x = 10; function abc() {...}
// Bom
let idadeUsuario = 10; function calcularTotal() {...}
```

2. Evite Variáveis Globais:
Variáveis globais podem causar colisões de nomes e tornar o código propenso a erros. Prefira encapsular variáveis em escopos locais, utilizando boas práticas de modularização.

```
// Ruim
let globalVar = 5; function soma() { return globalVar + 10; }
// Bom
function soma(localVar) { return localVar + 10; }
```

3. Use "const" e "let" Adequadamente:
Prefira const para valores que não serão reatribuídos e let para variáveis que precisam de reatribuição. Isso melhora a legibilidade e previne erros inadvertidos.

```
// Ruim
var total = 100;
// Bom
const limite = 100; let contador = 0;
```

4. Evite o Uso Excessivo de Aninhamento:
Aninhamento excessivo pode tornar o código difícil de entender. Utilize controle de fluxo apropriado, como o uso de return em funções, para evitar aninhamentos profundos.

```
// Ruim
if (condicao1) {
if (condicao2) {
  // ...
}
}
```

```
// Bom
if (!condicao1) return;
if (!condicao2) return;
// ...
```

BOAS PRÁTICAS

5. Utilize Arrow Functions e Template Strings:
Arrow functions e template strings podem tornar o código mais conciso e legível.

```
// Antes
function saudacao(nome) {
return 'Olá, ' + nome + '!';
}

// Depois
const saudacao = nome => `Olá, ${nome}!`;
```

6. Manuseie Exceções Adequadamente:
Sempre envolva código suscetível a erros em blocos try-catch para evitar que erros não tratados interrompam a execução do programa.

```
try {
// Código propenso a erros
} catch (erro) {
console.error(erro);
}
```

7. Faça Uso de Módulos:
Divida o código em módulos para promover a reutilização e manutenção. Importe apenas o que é necessário para cada módulo.

```
// moduloA.js
export const soma = (a, b) => a + b;
// moduloB.js
import { soma } from './moduloA'; console.log(soma(5, 10));
```

Conclusão:
Adotar boas práticas em JavaScript é fundamental para desenvolver código de qualidade, facilitando a colaboração, manutenção e reduzindo a probabilidade de bugs. Além disso, seguindo estas diretrizes, você estará alinhado com os padrões recomendados pela comunidade de desenvolvedores, contribuindo para um ecossistema JavaScript mais coeso e eficiente. Lembre-se de que essas práticas são flexíveis e podem ser adaptadas às necessidades específicas do seu projeto.

TIPOS DE DADOS

Tipagem dinâmica e fraca

JavaScript é uma linguagem dinâmica com tipos dinâmicos. As variáveis em JavaScript não estão diretamente associadas a nenhum tipo de valor específico, e qualquer variável pode receber (e reatribuir) valores de todos os tipos:

Variável	Explicação	Exemplo
String	Esta é uma sequência de texto conhecida como string. Para significar que o valor for uma string, coloque-a entre aspas simples.	`let myVariable = 'Bob';`
Number	Isto é um número. Os números não têm aspas.	`deixe minhaVariável = 10;`
Boolean	Este é um valor Verdadeiro/Falso. As palavras `true` e `false` são palavras-chave especiais que não precisam de aspas.	`let myVariable = true;`
Array	Esta é uma estrutura que permite armazenar vários valores em um único referência.	`let myVariable = [1,'Bob','Steve',10];` Refere-se a cada membro do array assim: `myVariable[0]`, `myVariable[1]`, etc.
Object	Isso pode ser qualquer coisa. Tudo em JavaScript é um objeto e pode ser armazenados em uma variável. Tenha isso em mente enquanto aprende.	`let myVariable = document.querySelector('h1');` Todos os exemplos acima também.

OBJETOS

Um objeto é uma coleção de dados e/ou funcionalidades relacionadas (que geralmente consistem em diversas variáveis e funções — que são chamadas de propriedades e métodos quando estão dentro de objetos). Vamos trabalhar com um exemplo para entender como eles são.

```
{
    nome: 'Ananias',
    idade: 20,
    endereco: {
        rua: "Vergueiro",
        cidade: "São Paulo"
        },
    telefones: ["91234-5678", '98765-4321'],
    "calcula-ano-nascimento": function(){
        //obtem ano atual
        var anoAtual = new Date().getFullYear();
         return anoAtual - this.idade;
    }
}
```

COMENTÁRIOS

```
/*
Tudo no meio é um comentário.
*/
```

```
// Isso é um comentário
```

OPERADORES

Um operator é um símbolo matemático que produz um resultado baseado em dois valores (ou variáveis). Na tabela a seguir, você pode ver alguns dos operadores mais simples, juntamente com alguns exemplos para experimentar no console JavaScript.

Operador	Explicação	Símbolo(s)	Exemplo
Adição	Adicione dois números ou combine duas strings.	`+`	`6 + 9;` `'Olá ' + 'mundo!';`

Subtração, Multiplicação, Divisão	Eles fazem o que você espera que façam em matemática básica.	`-`, `*`, `/`	`9 - 3;`
			`8 * 2; // multiplicar em JS é um asterisco`
			`9 / 3;`

Atribuição	Como você já viu: isso atribui um valor a uma variável.	`=`	`let myVariable = 'Bob';`
Igualdade estrita	Isso executa um teste para ver se dois valores são iguais. Ele retorna um Resultado `true` / `false` (booleano).	`===`	`let myVariable = 3;` `myVariable === 4;`

Não, não é igual	Isso retorna o valor logicamente oposto do que precede. Acontece um `true` em um `false`, etc. Quando é usado ao lado do operador de igualdade, o operador de negação testa se dois os valores *não* são iguais.	`!`, `!==`	Para "Not", a expressão básica é `true`, mas o a comparação retorna `false` porque nós a negamos:
			`let myVariable = 3;`
			`!(myVariable === 3);`
			"Não é igual" dá basicamente o mesmo resultado com diferentes sintaxe. Aqui estamos testando "é `myVariable` NÃO igual a 3". Isso retorna `false` porque `myVariable` É igual a 3:
			`let myVariable = 3;`
			`myVariable !== 3;`

OPERADORES MATEMÁTICOS

Um operator é um símbolo matemático que produz um resultado baseado em dois valores (ou variáveis). Na tabela a seguir, você pode ver alguns dos operadores mais simples, juntamente com alguns exemplos para experimentar no console JavaScript.

Operator	Name	Purpose	Example	Shortcut for
`+=`	Atribuição de adição	Adiciona o valor à direita ao valor da variável à esquerda e, em seguida, retorna o novo valor da variável	`x = 3; x += 4;`	`x = 3; x = x + 4;`
`-=`	Atribuição de subtração	Subtrai o valor à direita do valor da variável à esquerda e retorna o novo valor da variável	`x = 6; x -= 3;`	`x = 6; x = x - 3;`
`*=`	Atribuição de multiplicação	Multiplica o valor da variável à esquerda pelo valor à direita e retorna o novo valor da variável	`x = 2; x *= 3;`	`x = 2; x = x * 3;`
`/=`	Atribuição de divisão	Divide o valor da variável à esquerda pelo valor à direita e retorna o novo valor da variável	`x = 10; x /= 5;`	`x = 10; x = x / 5;`

Operator	Name	Purpose	Example
`===`	Igualdade estrita	Verifica se o valor da esquerda e o da direita são idênticos entre si.	`5 === 2 + 4`
`!==`	Não-igualdade-estrita	Verifica se o valor da esquerda e o da direita **não** são idênticos entre si.	`5 !== 2 + 3`
`<`	Menor que	Verifica se o valor da esquerda é menor que o valor da direita.	`10 < 6`
`>`	Maior que	Verifica se o valor da esquerda é maior que o valor da direita.	`10 > 20`
`<=`	Menor ou igual que	Verifica se o valor da esquerda é menor que ou igual ao valor da direita.	`3 <= 2`
`>=`	Maior ou igual que	Verifica se o valor da esquerda é maior que ou igual ao valor da direita.	`5 >= 4`

MEUS CONTATOS:

27 99500-7495

https://beacons.ai/prismatech

producaoprismatech@gmail.com

Avenida Jerônimo Monteiro 145, Vitória

THANK
YOU